

Mix, Configuration and Releases — A Readable Companion to the Elixir Docs

Chapter 10 · checked against Elixir 1.20 · Sep 26, 2026

1. Why build time and run time must stay apart

An Elixir project goes through two separate lives. At **build time**, Mix compiles your code on a build machine, with that machine’s environment, and assembles a release. At **run time**, the release boots on a server, with a different environment, and no Mix at all.

Configuration bugs almost always come from code that runs in one life and assumes the other. A value read during compilation is frozen into the `.beam` file. A call to Mix works on your laptop and doesn’t exist in production. A dependency marked for development compiles fine in `mix test` and is missing from the release. The Erlang book covers applications and releases from the OTP side (chapter 7); this chapter covers the Elixir tooling on top of it.

2. The building blocks

Configuration files

File	Evaluated	Can read	Use for
<code>config/config.exs</code> (+ <code>dev.exs</code> , <code>test.exs</code> , <code>prod.exs</code>)	At build time, before compiling	Build machine’s environment	Settings that shape compilation; per-environment defaults
<code>config/runtime.exs</code>	Each time the system or release starts	The target machine’s environment	Secrets, URLs, ports, anything deployment-specific

The docs: `config.exs` “is read at build time, before we compile our application,” while `runtime.exs` “is read after our application and dependencies are compiled.”

Reading configuration

Function	When it reads	Notes
<code>Application.get_env/3</code> , <code>fetch_env!/2</code>	When called	Use inside functions for runtime values
<code>Application.compile_env/3</code> , <code>compile_env!/2</code>	At compile time	Mix records the value and raises at boot if runtime config differs
<code>System.get_env/1</code>	When called	Returns <code>nil</code> if unset

Function	When it reads	Notes
<code>System.fetch_env!/1</code>	When called	Raises if unset: use in <code>runtime.exs</code> for required values

Dependencies

Option	Effect
<code>only: [:dev, :test]</code>	Not fetched or compiled for other environments; absent from prod releases
<code>runtime: false</code>	Compiled, but the application is not started at runtime (for build tools)
<code>mix.lock</code>	Records exact versions; commit it so every build uses the same code

Releases

`mix release` builds a self-contained directory with your code, dependencies and the Erlang runtime. The docs state the rule plainly: release code “MUST NOT access Mix in any way, as Mix is a build tool and it is not available inside releases.”

3. How it actually works

Module bodies run at compile time

Everything at the top level of a module, including module attributes, runs when the module is compiled. `@timeout Application.get_env(:app, :timeout)` stores whatever the config said on the build machine. The docs warn that in module bodies “the application environment is not yet available,” and point to `compile_env/3` when a compile-time value really is intended.

compile_env checks itself

`Application.compile_env/3` records the value used during compilation, and at boot “Mix will... compare the compilation values with the runtime values whenever your system starts, raising an error in case they differ.” A mismatch fails loudly at startup instead of silently in production.

The application environment is global

Every application’s configuration lives in one global store per node. The docs recommend that libraries avoid it: “the application environment is effectively a global storage.” Two dependents of a library can’t configure it differently.

4. Plausible but wrong

4.1 Secrets in build-time config

```
# config/prod.exs
config :my_app, MyApp.Repo, url: System.get_env("DATABASE_URL")
```

This runs on the build machine. There, `DATABASE_URL` is usually unset, so the release ships with `nil`, or with the build machine's database URL baked in. Fix: move it to `config/runtime.exs` and use `System.fetch_env!("DATABASE_URL")` so a missing value stops the boot with a clear error.

4.2 Configuration frozen in a module attribute

```
defmodule MyApp.Client do
  @base_url Application.get_env(:my_app, :base_url)
  def get(path), do: HTTP.get(@base_url <> path)
end
```

`@base_url` is evaluated at compile time, so setting `base_url` in `runtime.exs` changes nothing. Fix: read it at runtime (`Application.fetch_env!(:my_app, :base_url)` inside the function). If it truly must be compile-time, use `Application.compile_env!/2`, which at least raises at boot when runtime config disagrees.

4.3 Calling Mix at runtime

```
def debug?, do: Mix.env() == :dev
```

It works in `iex -S mix` and in tests. In a release the `Mix` module doesn't exist, and the first call raises `UndefinedFunctionError`. Fix: decide at build time with configuration: `config :my_app, debug: true` in `dev.exs`, read with `Application.get_env/3`.

4.4 A dev-only dependency used in production code

```
{:jason, "~> 1.4", only: [:dev, :test]}

def to_json(data), do: Jason.encode!(data)
```

Everything passes in development and CI. The prod release doesn't include `jason`, and the first call fails with `UndefinedFunctionError`. Fix: include every dependency that `lib/` code calls in all environments, and build and smoke-test the actual release in CI.

4.5 A library configured through the application environment

```
defmodule MyLib.Client do
  def request(path) do
    key = Application.fetch_env!(:my_lib, :api_key)
    ...
  end
end
```

```
end
end
```

Every application that depends on `my_lib` must share one global setting, so two services in one release can't use different keys, and tests can't run in parallel with different values. The design anti-patterns guide calls this *using application configuration for libraries*. Fix: accept options in function calls or `start_link/1`, and let the host application pass them from its own config.

4.6 Required values that default to nil

```
# config/runtime.exs
config :my_app, :payment_key, System.get_env("PAYMENT_KEY")
```

A typo in the variable name or a missing deployment secret gives `nil`. The system boots, reports healthy, and fails at the first payment. Fix: `System.fetch_env!/1` for required values, and parse types here too (`String.to_integer/1` for ports, explicit `== "true"` for flags, chapter 1).

4.7 An uncommitted lock file

```
.gitignore:
mix.lock
```

Without the lock file, each build resolves dependency versions afresh. The laptop, CI and production can all run different versions of the same library, and a patch release upstream changes your system without a commit. Fix: commit `mix.lock`; update dependencies deliberately with `mix deps.update` and review the diff.

5. Review checklist and sources

- ☐ Everything environment-specific is read in `config/runtime.exs`
- ☐ Required values use `System.fetch_env!/1`; types are parsed at startup
- ☐ No `Application.get_env` in module attributes; `compile_env` is used where compile-time values are intended
- ☐ No `Mix` calls in `lib/` code
- ☐ Dependencies used by `lib/` code are available in all environments
- ☐ Libraries take configuration as options, not from the application environment
- ☐ `mix.lock` is committed
- ☐ CI builds and boots the release, not only `mix test`

Sources

- Configuration and releases: <https://hexdocs.pm/elixir/config-and-releases.html>
- Application: <https://hexdocs.pm/elixir/Application.html>
- mix release: <https://hexdocs.pm/mix/Mix.Tasks.Release.html>
- Design-related anti-patterns: <https://hexdocs.pm/elixir/design-anti-patterns.html>

- Erlang book, chapter 7 (Applications and Releases): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>